INDICE

1. Introducción

2. Objetivo

3. Situación problemática

4. Modelo de negocio

5. Diagrama Entidad Relación

6. Listado de tablas

7. Vistas

8. Funciones

9. Stored Procedures

10. Triggers

11. Scripts SQL

12. Informes generados

13. Herramientas utilizadas

14. Conclusión

TP FINAL:

Idea: Distribuidora artículos consumo masivos

Alumno: Ale Bertochi

1. Introducción – Descripción de la temática

El presente proyecto consiste en el diseño y desarrollo de una base de datos relacional orientada a la gestión de una distribuidora de artículos de consumo masivo.
La base de datos tiene como finalidad centralizar la información vinculada a clientes, artículos, vendedores y pedidos, permitiendo registrar las operaciones comerciales de manera ordenada, consistente y segura.

2. Objetivo del proyecto

El objetivo del proyecto es implementar una base de datos que permita administrar las operaciones principales de una distribuidora, facilitando el registro de pedidos, el control de los artículos comercializados y el seguimiento de las ventas realizadas por cada vendedor y cliente.

Asimismo, la base de datos busca servir como fuente de información para el análisis comercial, contable, impositivo y logístico, permitiendo obtener reportes de ventas, comportamiento de clientes y rendimiento de los vendedores.

3. Situación Problemática

En la distribuidora, la gestión de la información se realiza de manera descentralizada, utilizando planillas de cálculo y registros manuales para el seguimiento de clientes, artículos y pedidos.

Esta forma de trabajo genera inconsistencias en los datos, duplicación de información y dificultades para mantener un historial confiable de las operaciones realizadas. Asimismo, no se cuenta con una herramienta que permita analizar de forma eficiente las ventas por cliente, por vendedor o por período, lo que limita la toma de decisiones y el control de la gestión.

Ante esta situación, surge la necesidad de implementar una base de datos centralizada que permita organizar la información, garantizar la integridad de los datos y facilitar el acceso a información confiable para el análisis operativo, contable, impositivo y logístico.

4. Modelo de Negocio

La distribuidora cuenta con vendedores responsables de gestionar los pedidos, los cuales son realizados por los clientes y pueden incluir uno o varios artículos. Cada artículo pertenece a una categoría que permite su clasificación, y los pedidos quedan registrados con su correspondiente fecha y responsable comercial.

Toda esa información se centraliza luego en el área de administración, la cual es el área troncal del negocio.

Este modelo permite representar de manera estructurada el flujo de la empresa, desde la oferta de artículos hasta la concreción de las ventas, facilitando el control y análisis de la información generada.

5. Identificación de Entidades

Entidades identificadas en el modelo

A partir del análisis del modelo de negocio, se identifican las siguientes entidades principales que conforman la base de datos de la distribuidora:

1. Clientes
2. Vendedores
3. Artículos
4. Categorías
5. Proveedores
6. Pedidos
7. Detalle_Pedido
8. Formas_Pago
9. Estados_Pedido
10. Localidades

Cada una de estas entidades representa un objeto relevante dentro del funcionamiento de la organización y será desarrollada en el modelo entidad–relación y en el esquema de la base de datos.

5.1 – Relaciones entre entidades

- Cliente 1 — N Pedido
- Cliente N — 1 Localidad
- Vendedor 1 — N Pedido
- Pedido N — 1 Forma_Pago
- Pedido N — 1 Estado_Pedido
- Pedido 1 — N Detalle_Pedido
- Artículo 1 — N Detalle_Pedido
- Categoría 1 — N Artículo
- Proveedor 1 — N Artículo

6. Diagrama Entidad Relación (DER)

6.1 Atributos principales por entidad

1. Clientes

- **id_cliente (PK)**
- nombre
- apellido_razon_social
- telefono
- email
- id_localidad (FK)

2. Vendedores

- **id_vendedor** (PK)
- nombre
- apellido
- teléfono
- mail

3. Artículos

- **id_articulo (PK)**
- nombre
- precio
- stock
- id_categoria (FK)
- id_proveedor (FK)

4. Categorías

- **id_categoria** (PK)
- nombre categoría
- descripción

5. Proveedores

- **id_proveedor** (PK)
- nombre
- teléfono
- email

6. Pedidos

- **id_pedido** (PK)
- fecha_pedido
- id_cliente (FK)
- id_vendedor (FK)
- id_forma_pago (FK)
- id_estado_pedido (FK)
- total

7. Detalle_Pedido

- id_detalle_pedido (PK)
- id_pedido (FK)
- id_articulo (FK)
- cantidad
- precio_unitario

8. Estados_Pedido

- **id_estado_pedido** (PK)
- descripción

9. Localidades

- **id_localidad** (PK)
- nombre_localidad
- provincia

10. Formas_Pago

- **id_forma_pago** (PK)
- descripción

6.2 Texto explicativo

El diagrama entidad–relación representa la estructura lógica de la base de datos propuesta, mostrando las entidades identificadas y las relaciones existentes entre ellas. Dicho diagrama se encuentra alineado con el modelo de negocio planteado y constituye la base para la posterior implementación del esquema físico mediante scripts SQL.

6.3 Características técnica de las tablas

CLIENTES

Campo	Abrev.	Descripción	Tipo de dato	Clave
id_cliente	id_cli	Identificador del cliente	INT	PK
nombre	nom_cli	Nombre del cliente	VARCHAR(50)	
apellido_razon_social	ape_rs	Apellido o razón social	VARCHAR(50)	
teléfono	tel_cli	Teléfono de contacto	VARCHAR(20)	
email	mail_cli	Correo electrónico	VARCHAR(50)	
id_localidad	id_loc	Localidad del cliente	INT	FK

LOCALIDADES

Campo	Abrev.	Descripción	Tipo de dato	Clave
id_localidad	id_loc	Identificador de la localidad	INT	PK
nombre_localidad	nom_loc	Nombre de la localidad	VARCHAR(500)	
provincia	prov	Provincia	VARCHAR(50)	

VENDEDORES

Campo	Abrev.	Descripción	Tipo de dato	Clave
id_vendedor	id_ven	Identificador del vendedor	INT	PK
nombre	nom_ven	Nombre del vendedor	VARCHAR(50)	
apellido	ape_ven	Apellido del vendedor	VARCHAR(50)	
telefono	tel_ven	Teléfono	VARCHAR(20)	
mail	mail_ven	Mail del vendedor	VARCHAR(50), INT	

CATEGORIAS

Campo	Abrev.	Descripción	Tipo de dato	Clave
id_categoria	id_cat	Identificador de la categoría	INT	PK
nombre_categoria	nom_cat	Nombre de la categoría	VARCHAR(50)	
descripcion	desc_cat	Descripción	VARCHAR(100)	

PROVEEDORES

Campo	Abrev.	Descripción	Tipo de dato	Clave
id_proveedor	id_prov	Identificador del proveedor	INT	**PK**
nombre	nom_prov	Nombre del proveedor	VARCHAR(50)	
teléfono	tel_prov	Teléfono	VARCHAR(20)	
email	mail_prov	Correo electrónico	VARCHAR(50)	

ARTICULOS

Campo	Abrev.	Descripción	Tipo de dato	Clave
id_articulo	id_art	Identificador del artículo	INT	PK
nombre	nom_art	Nombre del artículo	VARCHAR(100)	
precio	prec_art	Precio unitario	DECIMAL(10,2)	
stock	stk_art	Stock disponible	INT	

Campo	Abrev.	Descripción	Tipo de dato	Clave
id_categoria	id_cat	Categoría del artículo	INT	FK
id_proveedor	id_prov	Proveedor del artículo	INT	FK

MEDIOS DE PAGO

Campo	Abrev.	Descripción	Tipo de dato	Clave
id_forma_pago	id_fp	Identificador forma de pago	INT	PK
descripcion	desc_fp	Descripción	VARCHAR(50)	

PEDIDOS

Campo	Abrev.	Descripción	Tipo de dato	Clave
id_pedido	id_ped	Identificador del pedido	INT	PK
fecha_pedido	fec_ped	Fecha del pedido	DATE	
id_cliente	id_cli	Cliente	INT	FK
id_vendedor	id_ven	Vendedor	INT	FK
id_forma_pago	id_fp	Forma de pago	INT	FK
id_estado_pedido	id_ep	Estado del pedido	INT	FK
total	tot_ped	Total del pedido	DECIMAL(12,2)	

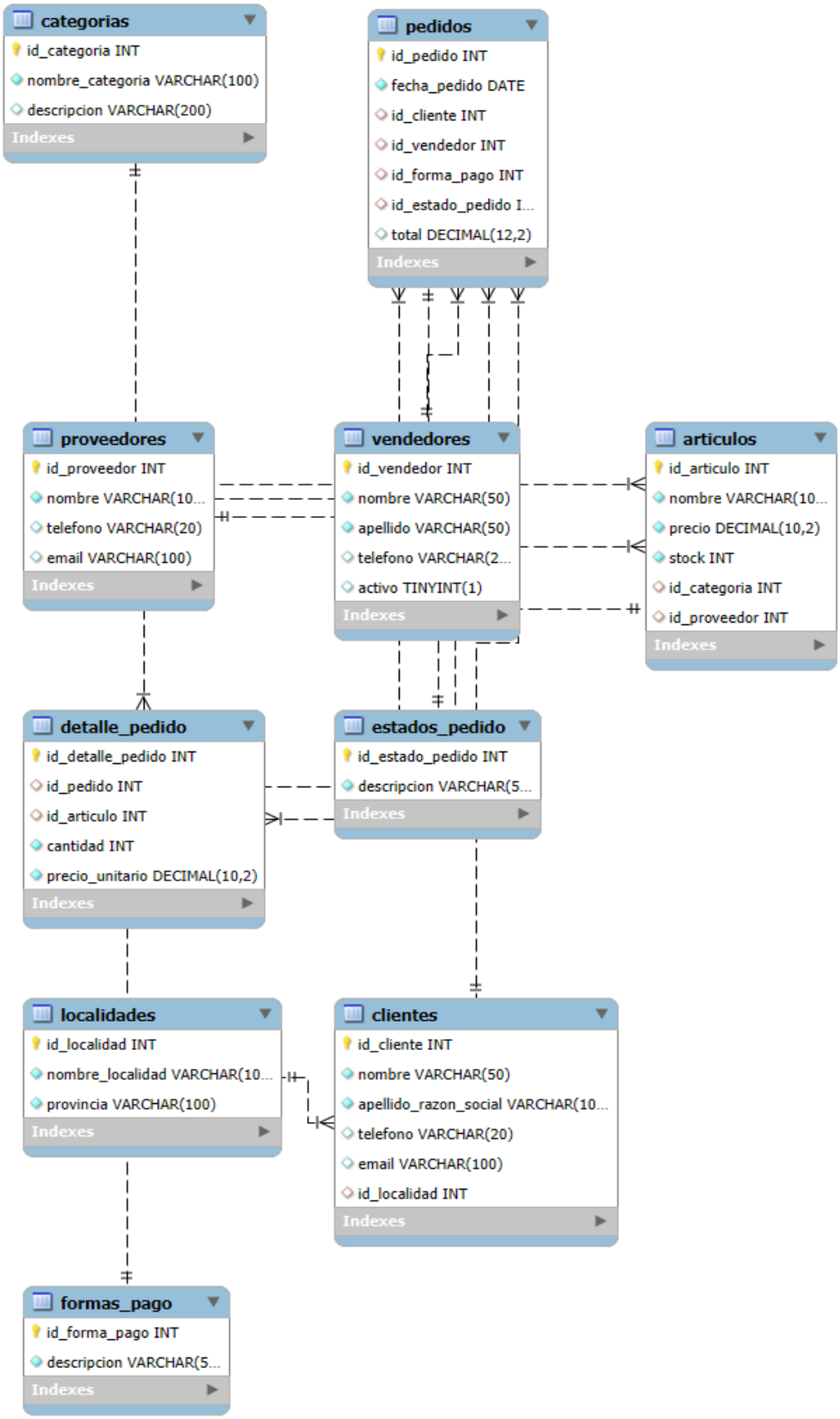
DETALLE DEL PEDIDO

Campo	Abrev.	Descripción	Tipo de dato	Clave
id_detalle_pedido	id_det	Identificador del detalle	INT	PK
id_pedido	id_ped	Pedido	INT	FK
id_articulo	id_art	Artículo	INT	FK
cantidad	cant	Cantidad	INT	
precio_unitario	prec_unit	Precio unitario	DECIMAL(10,2)	

ESTADO DE PEDIDOS

Campo	**Abrev.**	**Descripción**	**Tipo de dato**	**Clave**
id_estado_pedido	id_ep	Identificador del estado	INT	PK
descripción	desc_ep	Descripción del estado	VARCHAR(50)	

6.5 Grafico E- R



7. LISTADO DE VISTAS

En esta sección se detallan las vistas creadas en la base de datos “distribuidora”, indicando su objetivo y las tablas que intervienen en su construcción.

1. Vista: vw_detalle_pedidos

Descripción:

La vista vw_detalle_pedidos permite visualizar el detalle completo de cada pedido realizado en el sistema. Muestra información del pedido, del cliente y de los artículos incluidos, junto con la cantidad solicitada, el precio unitario y el subtotal calculado.

Objetivo

Su objetivo es simplificar la consulta de información relacionada a los pedidos, evitando realizar múltiples JOIN manualmente cada vez que se necesite analizar ventas. Facilita la generación de reportes comerciales y el control de operaciones.

Tablas involucradas

pedidos

clientes

detalle_pedido

articulos

2. Vista: vw_stock_articulos

Descripción:

La vista vw_stock_articulos muestra el stock actual de cada artículo junto con la categoría a la que pertenece.

Objetivo

Permite tener una visualización rápida del inventario disponible, facilitando el control de stock y la toma de decisiones relacionadas con reposición o planificación de compras.

Tablas involucradas

articulos

categorias

3. Vista: vw_total_pedidos_por_cliente

Descripción:

La vista vw_total_pedidos_por_cliente calcula el total acumulado de compras realizadas por cada cliente, sumando los subtotales de los artículos incluidos en sus pedidos.

Objetivo

Su finalidad es permitir el análisis del comportamiento de compra de los clientes, identificando aquellos con mayor volumen de operaciones y facilitando la gestión comercial.

Tablas involucradas

clientes

pedidos

detalle_pedido

Comentario final

Las vistas fueron diseñadas para centralizar consultas frecuentes y mejorar la legibilidad del sistema, evitando la repetición de consultas complejas y favoreciendo una estructura organizada de la base de datos.

8. LISTADO DE FUNCIONES

Se detallan las funciones creadas en la base de datos “distribuidora”, indicando su finalidad y las tablas sobre las que operan.

1. Función: fn_calcular_subtotal

Descripción:

La función fn_calcular_subtotal recibe como parámetros una cantidad y un precio unitario, y devuelve el resultado de la multiplicación entre ambos valores.

Objetivo

Su objetivo es encapsular el cálculo del subtotal de un artículo dentro de un pedido, evitando repetir la operación matemática en múltiples consultas y mejorando la claridad del código SQL.

Tablas involucradas

No modifica tablas directamente.
Se utiliza en conjunto con los datos de la tabla:

- detalle_pedido

2. Función: fn_total_pedido

Descripción:

La función fn_total_pedido recibe como parámetro el identificador de un pedido y devuelve el total del mismo, calculado como la suma de los subtotales de todos los artículos asociados.

Internamente realiza una consulta sobre los registros de detalle_pedidos correspondientes al pedido indicado.

Objetivo

Permitir obtener el total de un pedido de manera automática y centralizada, garantizando consistencia en los cálculos y facilitando su reutilización en consultas o reportes.

Tablas involucradas

- detalle_pedido

3. Función: fn_stock_disponible

Descripción:

La función fn_stock_disponible recibe como parámetro el identificador de un artículo y devuelve el stock actual registrado en la base de datos.

Objetivo

Facilitar la consulta del stock disponible, pudiendo ser utilizada para validaciones o controles dentro de procedimientos almacenados.

Tablas involucradas

- articulos

Comentario final:

Las funciones fueron creadas con el objetivo de encapsular cálculos frecuentes dentro del sistema, promoviendo reutilización, claridad y mantenimiento simplificado del código.

9. LISTADO DE STORED PROCEDURES

Se detallan los procedimientos almacenados creados en la base de datos “distribuidora”, indicando su objetivo y las tablas sobre las cuales interactúan.

1. Stored Procedure: sp_actualizar_stock

Descripción:

El procedimiento almacenado sp_actualizar_stock permite modificar el stock de un artículo determinado, recibiendo como parámetros el identificador del artículo y el nuevo valor de stock.

Realiza una actualización directa sobre la tabla articulos.

Objetivo

Centralizar la lógica de actualización de stock en un único punto controlado, evitando modificaciones manuales directas sobre la tabla y favoreciendo una estructura más organizada del sistema.

Tablas involucradas

- artículos

2. Stored Procedure: sp_reporte_pedidos_por_fecha

Descripción

El procedimiento sp_reporte_pedidos_por_fecha permite obtener un listado de pedidos realizados dentro de un rango de fechas determinado. Recibe como parámetros una fecha de inicio y una fecha de fin.

Devuelve todos los pedidos cuya fecha se encuentre dentro del rango especificado.

Objetivo

Facilitar la generación de reportes comerciales y análisis de ventas en períodos específicos, permitiendo filtrar información sin necesidad de escribir consultas complejas cada vez.

Tablas involucradas

- pedidos

Comentario final:

Los Stored Procedures fueron implementados para encapsular operaciones frecuentes del sistema, promoviendo mayor orden, reutilización y control sobre las modificaciones y consultas realizadas en la base de datos.

10. LISTADO DE TRIGGERS

Se detallan los triggers implementados en la base de datos “distribuidora”, indicando su funcionalidad, el momento en que se ejecutan y las tablas involucradas.

1. Trigger: tr_descontar_stock

Descripción:

El trigger tr_descontar_stock se ejecuta automáticamente después de que se inserta un registro en la tabla detalle_pedido (evento AFTER INSERT).

Su función es descontar del stock del artículo la cantidad correspondiente al nuevo detalle de pedido ingresado.

Momento de ejecución

- AFTER INSERT

Sobre la tabla detalle_pedido

Objetivo

Garantizar que el stock se actualice automáticamente cada vez que se registra un artículo en un pedido, manteniendo la coherencia entre ventas e inventario sin depender de actualizaciones manuales.

Tablas involucradas

- detalle_pedido
- articulos

2. Trigger: tr_evitar_stock_negativo

Descripción:

El trigger tr_evitar_stock_negativo se ejecuta antes de que se actualice un registro en la tabla articulos (evento BEFORE UPDATE).

Verifica que el nuevo valor de stock no sea menor a cero. En caso contrario, genera un error y cancela la operación.

Momento de ejecución

- BEFORE UPDATE
- Sobre la tabla articulos

Objetivo

Evitar inconsistencias en el sistema, asegurando que el stock de un artículo nunca quede en valores negativos, lo cual representaría una situación inválida en términos comerciales.

Tablas involucradas

- articulos

Comentario final

Los triggers fueron implementados para reforzar la integridad de los datos y automatizar comportamientos clave del sistema, como la actualización del stock y el control de valores inválidos.

11. SCRIP SQL ENTREGADOS

Se incluyen tres archivos en formato .sql, organizados de manera separada para mantener una estructura clara proyecto.

1. 01_Creacion_de_Tablas

Contenido:

El presente script SQL contiene las sentencias necesarias para la creación del esquema de base de datos de la distribuidora.

En primer lugar, se crea la base de datos denominada distribuidora y luego se selecciona para su utilización mediante la sentencia USE.

Posteriormente se crean las diferentes tablas que componen el modelo de datos del sistema:

- localidades
- clientes
- vendedores
- categorias
- proveedores
- articulos
- formas_pago
- estados_pedido
- pedidos
- detalle_pedido

Cada tabla incluye:

- definición de sus campos o columnas
- el tipo de dato correspondiente
- la clave primaria (PRIMARY KEY)
- las claves foráneas (FOREIGN KEY) necesarias para establecer las relaciones entre las entidades.

Estas relaciones permiten vincular correctamente la información entre clientes, vendedores, artículos y pedidos, garantizando la integridad referencial de los datos.

Objetivo:

El objetivo de este script es crear la estructura física de la base de datos que soporta el funcionamiento del sistema de gestión de la distribuidora.

Mediante la definición de tablas y relaciones se establece un modelo de datos que permite:

- almacenar información de clientes, vendedores y proveedores
- registrar los artículos disponibles y su stock
- gestionar los pedidos realizados por los clientes
- registrar el detalle de los artículos incluidos en cada pedido

- identificar formas de pago y estados de los pedidos

De esta manera, la base de datos permite organizar la información del negocio de forma estructurada, garantizando la consistencia de los datos y facilitando posteriormente la generación de consultas, reportes y análisis comerciales.

2. 02_creacion_objetos_distribuidora.sql

Contenido:

Este archivo contiene:

- Sentencias CREATE VIEW
- Sentencias CREATE FUNCTION
- Sentencias CREATE PROCEDURE
- Sentencias CREATE TRIGGER

Antes de la creación de los objetos se incluye la sentencia:

USE distribuidora;

Además, se incorporan sentencias DROP IF EXISTS para permitir que el script pueda ejecutarse múltiples veces sin generar errores por objetos ya existentes.

Objetivo:

Centralizar toda la lógica del sistema (consultas reutilizables, cálculos, automatizaciones y validaciones) en un único script ejecutable de forma independiente.

Este archivo crea la estructura lógica del sistema, pero no inserta datos.

3. 03_insercion_datos_distribuidora.sql

Contenido

Este archivo contiene sentencias INSERT INTO para todas las tablas del sistema:

- localidades
- categorias
- proveedores
- vendedores
- clientes
- estados_pedido
- formas_pago
- articulos

- pedidos
- detalle_pedido

También incluye la sentencia:

USE distribuidora;

Objetivo:

Permitir la carga inicial de datos de prueba, necesarios para verificar el funcionamiento de:

- Vistas
- Funciones
- Stored Procedures
- Triggers

Este archivo permite demostrar que el sistema funciona correctamente en un entorno real.

ORDEN DE EJECUCIÓN

Para garantizar el correcto funcionamiento del sistema, los scripts deben ejecutarse en el siguiente orden:

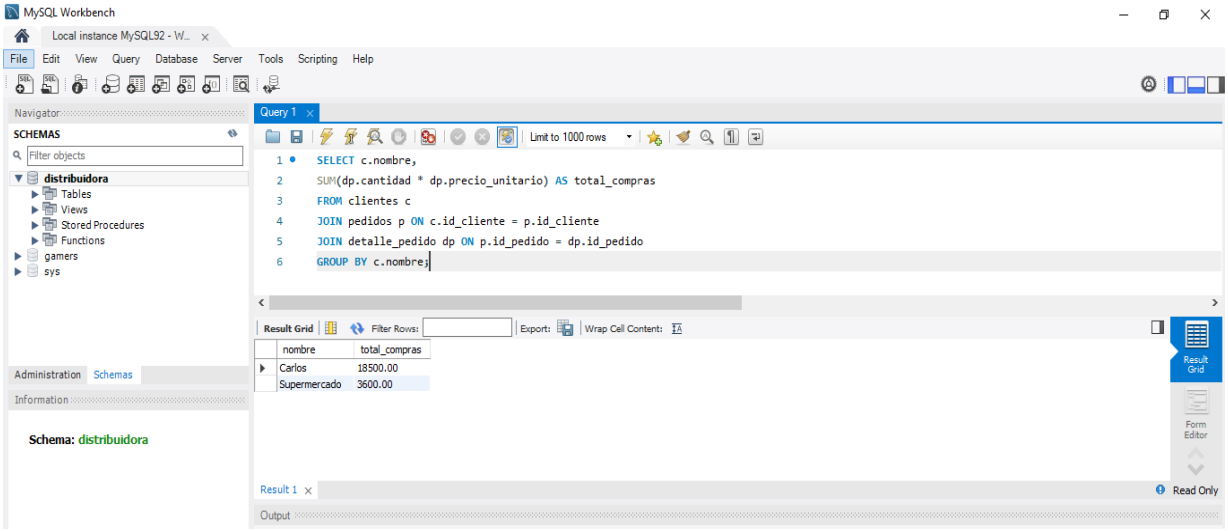
1. 01.Creacion_de_Tablas.sql
2. 02_creacion_objetos_distribuidora.sql
3. 03_insercion_datos_distribuidora.sql

De esta manera se crearon las tablas, luego se crean los objetos lógicos del sistema y luego se cargan los datos para su prueba y validación.

12. INFORMES GENERADOS

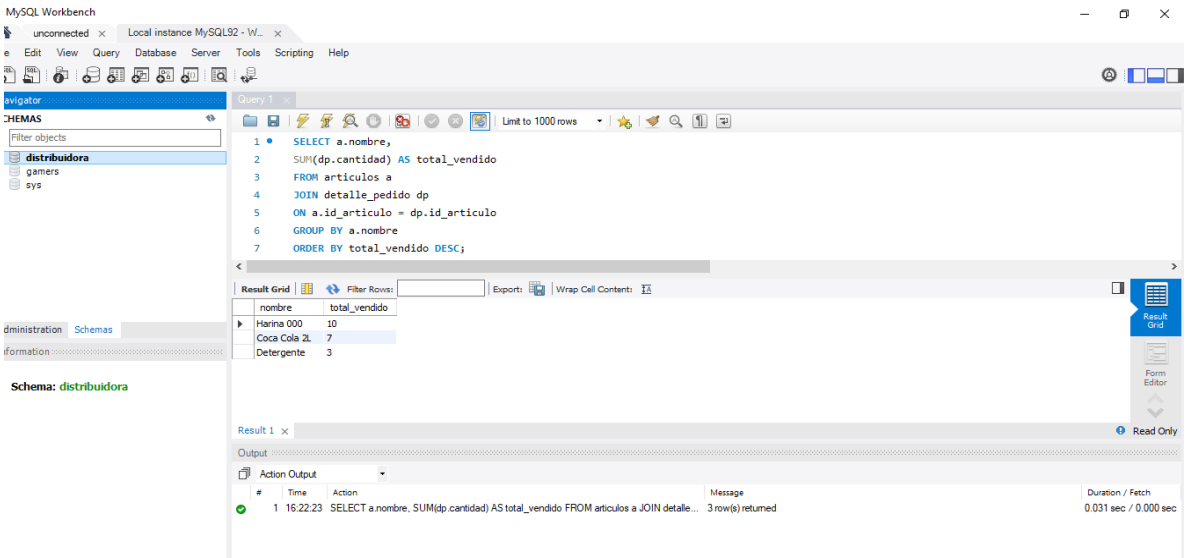
Informe 1

Ventas por cliente



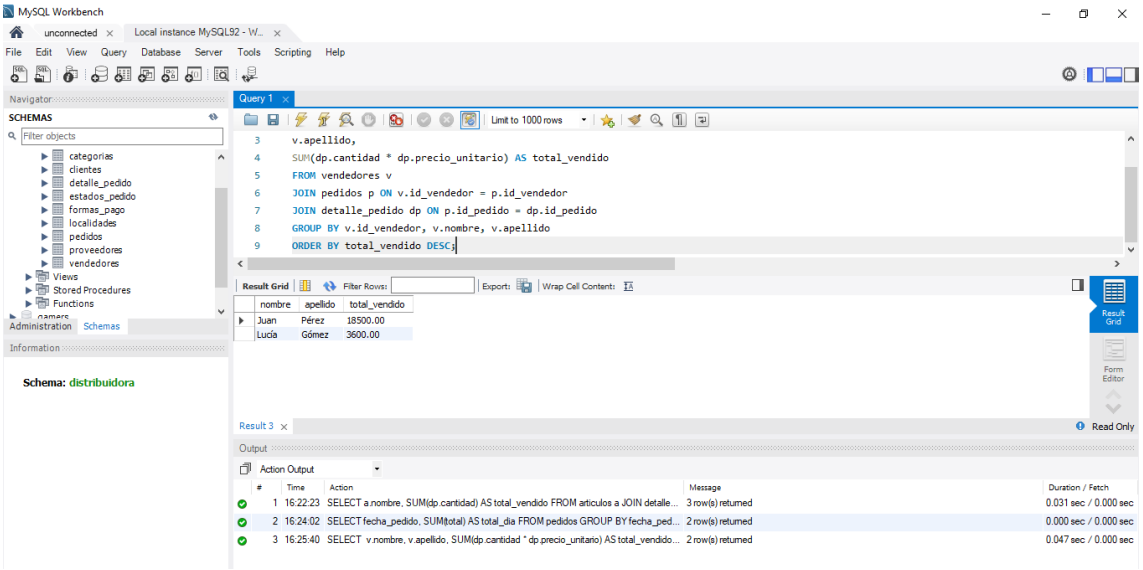
Informe 2

Artículos más vendidos



Informe 3

Ventas por Vendedor



13. Herramientas y tecnología utilizada

1. MySQL

2. MySQL Workbench

3. SQL

4. GitHub

5. Microsoft Excel (para análisis de datos)
6. ChatGPT

14. CONCLUSIÓN DEL PROYECTO

El presente proyecto permitió desarrollar una base de datos relacional para la gestión de una distribuidora, contemplando la administración de clientes, artículos, pedidos, vendedores y proveedores.

Durante esta segunda etapa se incorporaron objetos avanzados de base de datos, tales como vistas, funciones, procedimientos almacenados y triggers, con el objetivo de mejorar la organización, reutilización y control de la información.

Se logró:

- Automatizar la actualización del stock.

- Evitar inconsistencias como stock negativo.
- Centralizar cálculos de totales.
- Simplificar consultas mediante vistas.
- Facilitar reportes por rango de fechas.

El sistema implementado cumple con los requisitos funcionales y técnicos solicitados, manteniendo una estructura clara, coherente y normalizada.